

1. Introduction. Algorithm X, in §7.2.2.1 of *The Art of Computer Programming*, solves exact cover problems by backtracking. At step X3 it looks over the items not yet covered and picks one whose **LEN**—the number of options still available to it—is least. That is the *MRV* heuristic, minimum remaining values. Each option covering the chosen item then becomes a branch. So the shape of the search tree is settled entirely by which item wins the **LEN** contest at every step, and the cost of a run is the size of that tree.

Exercises 29 and 30 turn the arrow around. Instead of asking what tree a problem produces, they ask what problem produces a given tree.

29. [26] Let T be any tree. Construct the 0–1 matrix of an unsolvable exact cover problem for which T is the backtrack tree traversed by Algorithm X with the MRV heuristic. (A *unique* item should have the minimum **LEN** value whenever step X3 is encountered.) Illustrate your construction when T is a root with two subtrees, each having three leaves.

30. [25] Continuing exercise 29, let T be a tree in which certain leaves have been distinguished from the others and designated as “solutions.” Can all such trees arise as backtrack trees in Algorithm X?

2. Knuth answers yes to both, by explicit construction. This program checks him.

I wrote it in September 2026 while auditing the two exercises, and it turned up one thing worth reporting: answer 30, as printed in the 2022 first edition, breaks on exactly one tree—the one-node tree whose root is marked a solution. Knuth had already caught that. His file of changes to Volume 4B carries the correction, dated 22 March 2023, and it does more than patch the corner: it deletes a special rule that the corrected base case now supplies for free. Both readings live here behind a flag, so the difference can be measured instead of argued about.

The audit itself, with the numbers this program produced, is the companion document `README.md` in the directory above.

3. Answer 29 builds the matrix by recursion on T . A leaf becomes one column with no rows—an item that no option can cover, so the search dies there at once. An interior node with subtrees $T_1, \dots, T_d$ takes the matrices already built for its subtrees, whose columns are $C_1, \dots, C_d$, sets $C = C_1 \cup \dots \cup C_d$, and appends three fresh columns named 0, 1, 2 together with $d + 2$ fresh rows:

- (i) ‘0 1 2 and all columns of $C \setminus C_j$ ’, for $1 \leq j \leq d$;
- (ii) ‘ j and all columns of C ’, for $j \in \{0, 1\}$.

Column 2 then has the strictly smallest **LEN**, namely d , so Algorithm X must branch there, on the d rows of type (i), one per subtree. Choosing the j th covers everything outside C_j and leaves exactly the matrix of T_j standing. The rows of type (ii) are never chosen at all; they exist only to lift columns 0 and 1 one notch above column 2, which is what makes the minimum unique.

4. So the program has four jobs. It builds the matrix from a tree; it runs a faithful Algorithm X on the result and records the tree that comes back; it generates trees to feed the first two, either at random or exhaustively; and it hands the matrix to our own `ssxcc` for a second opinion on the number of solutions. The skeleton is a command line and a choice of what to do with it.

```
package main
import (
    "flag"
    "fmt"
    "log"
    "math/rand"
    "os"
    "sort"
    "strings"
    cells "github.com/sjnam/dancing-cells"
)
<Declarations 6>
<Functions 7>
func main() {
    <Read the command line 5>
    switch *mode {
    case "book":
        <Reproduce the printed matrix 28>
    case "dlx":
        <Write the problem as DLX text 31>
    case "ssxcc":
        <Ask ssxcc for a second opinion 32>
    case "all":
        <Try every small tree 33>
    case "random":
        <Try random trees 36>
    default:
        log.Fatalf("unknown mode %q", *mode)
    }
}
```

5. The flag that matters most is `-errata`. Everything else adjusts how much work to do.

<Read the command line 5> $\equiv$

```
mode := flag.String("mode", "book", "book, dlx, ssxcc, all, or random")
spec := flag.String("spec", "", "tree to use, e.g. ((.*) (...))")
fix := flag.Bool("errata", false, "follow the 2023-03-22 erratum for answer 30")
upto := flag.Int("upto", 8, "mode all: try all trees up to this many nodes")
trials := flag.Int("trials", 2000, "mode random: how many trees")
budget := flag.Int("budget", 14, "mode random: largest tree to build")
sol := flag.Bool("sol", false, "mode random: designate solution leaves")
sibs := flag.Bool("sibs", true, "mode random: allow sibling solution leaves")
seed := flag.Int64("seed", 1, "mode random: seed")
flag.Parse()
errata = *fix
```

This code is used in section 4.

6. Trees. A tree is a node with children. One bit of decoration is all exercise 30 asks for: a leaf may be *designated* a solution, meaning that Algorithm X should find a solution when it arrives there instead of failing.

⟨Declarations 6⟩ ≡

```
type node struct {
    kids []*node
    sol bool // a leaf designated a “solution” in exercise 30
}
```

This code is also defined in sections 11, 12, 19.

This code is used in section 4.

7. Two methods travel with it. The size of a tree is what the node count of a run will be compared against, and the string form is how trees are written on the command line and reported when something goes wrong: a dot is an ordinary leaf, a star a designated one, and parentheses enclose the children of an interior node. The tree drawn in exercise 29 is ((...)(...)).

⟨Functions 7⟩ ≡

```
func (t *node) size() int {
    n := 1
    for _, k := range t.kids {
        n += k.size()
    }
    return n
}

func (t *node) String() string {
    if len(t.kids) == 0 {
        if t.sol {
            return "*"
        }
        return "."
    }
    var b strings.Builder
    b.WriteByte('(')
    for _, k := range t.kids {
        b.WriteString(k.String())
    }
    b.WriteByte(')')
    return b.String()
}
```

This code is also defined in sections 8, 9, 10, 13, 17, 18, 20, 25, 26, 27, 30.

This code is used in section 4.

8. Reading that notation back is a short recursive descent. With no argument we hand back the tree of exercise 29, since that is the one the book draws and the one worth looking at first—and writing the default as a spec rather than as nodes keeps the two subtrees from quietly sharing their leaves.

⟨Functions 7⟩ +≡

```

func tree(spec string) *node {
  if spec ≡ "" {
    spec = "((...)(...))" // the tree drawn in exercise 29
  }
  pos := 0
  var parse func() *node
  parse = func() *node {
    switch c := spec[pos]; c {
      case '.', '*', ' ':
        pos++
        return &node{sol: c ≡ '*'}
      case '(':
        pos++
        t := &node{}
        for spec[pos] ≠ ')' {
          t.kids = append(t.kids, parse())
        }
        pos++
        return t
      default:
        log.Fatalf("bad tree spec at position %d of %q", pos, spec)
        return Λ
    }
  }
  return parse()
}

```

9. For the random sweep I grow a tree inside a budget of nodes, spending it on a fan-out of one to three subtrees and recursing. The shapes this makes are lumpy and unbalanced, which is what I want—the balanced tree of exercise 29 is the one case a bug would be least likely to survive.

⟨Functions 7⟩ +≡

```

func randTree(rnd *rand.Rand, budget int) *node {
  if budget ≤ 1 ∨ rnd.Intn(3) ≡ 0 {
    return &node{}
  }
  t := &node{}
  d, rest := 1 + rnd.Intn(3), budget - 1
  for j := 0; j < d ∧ rest > 0; j++ {
    t.kids = append(t.kids, randTree(rnd, 1 + rnd.Intn(rest)))
    rest -= t.kids[len(t.kids) - 1].size()
  }
  if len(t.kids) ≡ 0 {
    return &node{}
  }
  return t
}

```

10. Designating solutions afterwards takes one pass. The *sibOK* switch is there because of a remark at the end of answer 30—that without duplicate options no two solution nodes can be siblings—and turning it off is how I looked for the duplicates that remark predicts.

⟨ Functions 7 ⟩ +≡

```

func markSolutions(t *node, rnd *rand.Rand, p float64, sibOK bool) {
  used := false
  for _, k := range t.kids {
    if len(k.kids) == 0 ∧ rnd.Float64() < p ∧ (sibOK ∨ ¬used) {
      k.sol, used = true, true
    }
    markSolutions(k, rnd, p, sibOK)
  }
}

```

11. The construction. A matrix is a column count and a list of rows, each row being the sorted indices of its ones. Columns are numbered as they are created, so a subtree’s columns come out contiguous and the printed matrix falls into the same blocks the book shows.

⟨Declarations 6⟩ +≡

```
type matrix struct {
    ncol int
    rows [][]int // each row is a sorted list of column indices
}
```

12. Two globals steer the recursion: the next column number to hand out, and which reading of answer 30 to follow. Both are set once per matrix and never touched during a search, so their being globals costs nothing and saves threading them through every call.

⟨Declarations 6⟩ +≡

```
var nextCol int
var errata bool // follow the erratum rather than the printed answer 30
```

13. Here is the recursion itself. The base case is the one line of answer 29—one column, no rows—except under the erratum, where a leaf marked as a solution gets no column either. That single early return is the whole of the correction, and §3.4 of the audit explains why it suffices: with C_j empty, $C \setminus C_j$ is C , which is exactly what the printed answer asked for by hand.

⟨Functions 7⟩ +≡

```
func build(t *node)(cols []int, rows [][]int) {
    if len(t.kids) ≡ 0 {
        if errata ∧ t.sol {
            return  $\Lambda$ ,  $\Lambda$  // “let there be no rows and no columns”
        }
        nextCol++
        return []int{nextCol − 1},  $\Lambda$ 
    }
    ⟨Recurse into the subtrees 14⟩
    ⟨Append the rows of type (i) 15⟩
    ⟨Append the rows of type (ii) 16⟩
    cols = append(C, p0, p1, p2)
    sort.Ints(cols)
    return cols, rows
}
```

14. The subtrees are built first, in order, so that their rows are already in the list before any of ours. Then come the three new columns.

⟨Recurse into the subtrees 14⟩ ≡

```

var sub [][]int // the columns  $C_j$  of each subtree
for _, k := range t.kids {
    cj, rj := build(k)
    sub = append(sub, cj)
    rows = append(rows, rj...)
}
var C []int
for _, cj := range sub {
    C = append(C, cj...)
}
p0, p1, p2 := nextCol, nextCol + 1, nextCol + 2
nextCol += 3

```

This code is used in section 13.

15. A row of type (i) takes the three new columns and everything in C that does not belong to its own subtree. The printed answer 30 makes its one exception right here, taking all of C when the subtree is a designated solution, which leaves nothing uncovered and so ends the search in success.

⟨Append the rows of type (i) 15⟩ ≡

```

for j := range t.kids {
    r := []int{p0, p1, p2}
    for _, c := range C {
        if (¬errata ∧ t.kids[j].sol) ∨ ¬contains(sub[j], c) {
            r = append(r, c)
        }
    }
    sort.Ints(r)
    rows = append(rows, r)
}

```

This code is used in section 13.

16. The two rows of type (ii) are the ballast. Neither contains column 2, so neither is ever chosen; they are in the matrix only to give columns 0 and 1 a LEN of $d + 1$ against column 2's d .

⟨Append the rows of type (ii) 16⟩ ≡

```

for _, p := range []int { p0, p1 } {
    r := append([]int{p}, C...)
    sort.Ints(r)
    rows = append(rows, r)
}

```

This code is used in section 13.

17. Membership in a short unsorted list is wanted in one place, but a Go **for** loop cannot answer a question with a value, so it is a function.

⟨ Functions 7 ⟩ +≡

```
func contains(xs []int, x int) bool {
    for _, y := range xs {
        if y == x {
            return true
        }
    }
    return false
}
```

18. Starting the recursion means resetting the column counter. Everything above this line is answer 29 and 30; everything below is the machinery for judging them.

⟨ Functions 7 ⟩ +≡

```
func makeMatrix(t *node) matrix {
    nextCol = 0
    _, rows := build(t)
    return matrix{ncol: nextCol, rows: rows}
}
```


19. Algorithm X, taken literally. Our own solvers are no use here. Both engines in this repository descend from Knuth's sparse-set programs, and `ssxcc` in particular carries two refinements that Algorithm X does not have: it collects items of size one and forces them before branching, and its *hide* refuses a branch the moment an active primary item would be starved of options. On the matrix of exercise 29 that second refinement finds each failure one level early, so `ssxcc` reports three nodes where Algorithm X walks nine. Neither number is wrong; they are different algorithms. To judge an exercise about Algorithm X I need Algorithm X.

So this is the textbook procedure with nothing added, on dense arrays, written for clarity rather than speed. Because it must report the shape of the search, each call returns the node it visited, with its children attached.

⟨Declarations 6⟩ +≡

```
type trace struct {
    nodes int    // calls made, which is the size of the backtrack tree
    solutions int // times the matrix emptied out
    ties int     // times step X3 found no unique minimum
    dups int     // duplicate rows in the matrix
    shape *node  // the backtrack tree that was traversed
}
```

20. The active columns are a bit vector and the active rows a list of indices. The invariant is the usual one: every active row lies wholly inside the active columns.

⟨Functions 7⟩ +≡

```
func algX(m matrix) *trace {
    tr := &trace{ }
    cols := make([]bool, m.ncol)
    for i := range cols {
        cols[i] = true
    }
    rows := make([]int, len(m.rows))
    for i := range rows {
        rows[i] = i
    }
    var rec func([]bool, []int) *node
    rec = func(cols []bool, rows []int) *node {
        tr.nodes ++
        self := &node{ }
        ⟨Visit a solution if nothing is left 21⟩
        ⟨Choose the item of minimum LEN 22⟩
        ⟨Branch on every option covering it 23⟩
        return self
    }
    tr.shape = rec(cols, rows)
    ⟨Count duplicate options 24⟩
    return tr
}
```

21. Step X2. An empty matrix is a solution, and the leaf we mark here is the one that must line up with a star in the tree we were given.

⟨ Visit a solution if nothing is left 21 ⟩ ≡

```

empty := true
for _, b := range cols {
    if b {
        empty = false
        break
    }
}
if empty {
    tr.solutions++
    self.sol = true
    return self
}

```

This code is used in section 20.

22. Step X3, the heart of the matter. Exercise 29 asks that a *unique* item attain the minimum every time we get here, so as well as finding the minimum I count how many columns share it. Any tie is a failure of the construction, and in the whole audit there was never one.

⟨ Choose the item of minimum LEN 22 ⟩ ≡

```

best, least, ties := -1, 1 << 30, 0
for c := range cols {
    if ¬cols[c] {
        continue
    }
    n := 0
    for _, r := range rows {
        if contains(m.rows[r], c) {
            n++
        }
    }
    switch {
    case n < least:
        best, least, ties = c, n, 1
    case n ≡ least:
        ties++
    }
}
if ties ≠ 1 {
    tr.ties++
}

```

This code is used in section 20.

23. Steps X4 and X5. Covering a row deletes its columns and every row that meets one of them; what survives is passed down. When the chosen column has no rows at all—the leaf case of the construction—this loop simply does nothing, and the node stands as a childless failure.

⟨ Branch on every option covering it 23 ⟩ ≡

```

for _, r := range rows {
  if ¬contains(m.rows[r], best) {
    continue
  }
  nc := make([]bool, len(cols))
  copy(nc, cols)
  hit := make([]bool, len(cols))
  for _, x := range m.rows[r] {
    nc[x], hit[x] = false, true
  }
  var nr []int
  for _, r2 := range rows {
    clash := false
    for _, x := range m.rows[r2] {
      if hit[x] {
        clash = true
        break
      }
    }
    if ¬clash {
      nr = append(nr, r2)
    }
  }
  self.kids = append(self.kids, rec(nc, nr))
}

```

This code is used in section 20.

24. Answer 30 grants itself duplicate options, and remarks that without them no two solution nodes could be siblings. That is true of exact cover in general: if two options each finish a solution outright, each covers every item still remaining, and a surviving option lies wholly within the remaining items, so the two options are the same set. Counting duplicates lets me watch the remark come true.

⟨ Count duplicate options 24 ⟩ ≡

```

seen := map[string]bool{ }
for _, r := range m.rows {
  if k := fmt.Sprintf(r); seen[k] {
    tr.dups++
  } else {
    seen[k] = true
  }
}

```

This code is used in section 20.

25. Judging the result. A run passes when the tree that came back is the tree that went in—same shape, and the same leaves marked as solutions. Ordered trees, so no canonical form is needed; the children come back in the order their rows were laid down.

⟨ Functions 7 ⟩ +≡

```

func sameShape(a, b *node) bool {
  if len(a.kids) ≠ len(b.kids) {
    return false
  }
  if len(a.kids) ≡ 0 ∧ a.sol ≠ b.sol {
    return false
  }
  for i := range a.kids {
    if ¬sameShape(a.kids[i], b.kids[i]) {
      return false
    }
  }
  return true
}

```

26. Three more walks. The leaves of a tree are wanted both for counting the solutions we expect and for setting the marks in the exhaustive sweep; a copy is wanted so that one tree can be marked many ways; and the sibling test is the other half of the duplicate-option remark.

⟨ Functions 7 ⟩ +≡

```

func leaves(t *node) []*node {
  if len(t.kids) ≡ 0 {
    return []*node{t}
  }
  var out []*node
  for _, k := range t.kids {
    out = append(out, leaves(k)...)
  }
  return out
}

func clone(t *node) *node {
  c := &node{sol: t.sol}
  for _, k := range t.kids {
    c.kids = append(c.kids, clone(k))
  }
  return c
}

func hasSiblingSolutions(t *node) bool {
  n := 0
  for _, k := range t.kids {
    if len(k.kids) ≡ 0 ∧ k.sol {
      n++
    }
    if hasSiblingSolutions(k) {
      return true
    }
  }
  return n ≥ 2
}

```

27. For the exhaustive sweep I need every rooted ordered tree on n nodes. A tree is a root over a forest of $n - 1$ nodes, and a forest is a first tree of some size k followed by a forest of the rest, so the two recursions call each other. The counts are the Catalan numbers 1, 1, 2, 5, 14, 42, ..., which is why ten nodes is about as far as this goes in a comfortable minute.

⟨ Functions 7 ⟩ +≡

```

func allTrees(n int) []*node {
  if n ≡ 1 {
    return []*node{{ }}
  }
  var forests func(int) [][]*node
  forests = func(rest int) [][]*node {
    if rest ≡ 0 {
      return [][]*node{Λ}
    }
    var res [][]*node
    for k := 1; k ≤ rest; k++ {
      for _, first := range allTrees(k) {
        for _, tail := range forests(rest - k) {
          res = append(res, append([]*node{first}, tail...))
        }
      }
    }
    return res
  }
  var out []*node
  for _, f := range forests(n - 1) {
    out = append(out, &node{kids: f})
  }
  return out
}

```

28. Running it. The default mode rebuilds the figure on page 421. Answer 29 says the matrix for the example tree has 15 columns and 14 rows, and prints it; this is where I found that it does, to the bit.

⟨Reproduce the printed matrix 28⟩ ≡

```
t := tree(*spec)
m := makeMatrix(t)
fmt.Printf("tree\s:\td\nodes\n", t, t.size())
fmt.Printf("matrix:\td\columns,\td\rows\n", m.ncol, len(m.rows))
if m.ncol ≤ 40 {
    ⟨Print the matrix as an array of bits 29⟩
}
tr := algX(m)
fmt.Printf("Algorithm_X:\td\nodes,\td\solutions,\td\non-unique_minima\n",
    tr.nodes, tr.solutions, tr.ties)
fmt.Printf("backtrack_tree=\s\n", tr.shape)
fmt.Printf("shape>equals_T:\tv\n", sameShape(t, tr.shape))
```

This code is used in section 4.

29. ⟨Print the matrix as an array of bits 29⟩ ≡

```
for _, r := range m.rows {
    line := make([]byte, m.ncol)
    for i := range line {
        line[i] = '0'
    }
    for _, c := range r {
        line[c] = '1'
    }
    fmt.Printf("\s\n", line)
}
```

This code is used in section 28.

30. The same matrix in DLX text, which is what both of our engines read. Every item is primary, so there is no vertical bar in the first line, and column *c* is called *cc*.

⟨Functions 7⟩ +≡

```
func dlxText(m matrix) string {
    var b strings.Builder
    for c := 0; c < m.ncol; c++ {
        fmt.Fprintf(&b, "c%d", c)
    }
    b.WriteByte('\n')
    for _, r := range m.rows {
        for _, c := range r {
            fmt.Fprintf(&b, "c%d", c)
        }
        b.WriteByte('\n')
    }
    return b.String()
}
```

31. $\langle$ Write the problem as DLX text 31 $\rangle \equiv$

```
os.Stdout.WriteString(dlxText(makeMatrix(tree(*spec))))
```

This code is used in section 4.

32. How many solutions a problem has does not depend on the algorithm that finds them, so `ssxcc` can check that half of the work even though its search tree is its own. In exercise 30 the answer should be the number of leaves we marked.

$\langle$ Ask `ssxcc` for a second opinion 32 $\rangle \equiv$

```
t := tree(*spec)
m := makeMatrix(t)
want := 0
for _, l := range leaves(t) {
    if l.sol {
        want++
    }
}
tr := algX(m)
fmt.Printf("tree\s:\s%d\nodes,\s%d\ndesignated\solution\sleaves\n",
    t, t.size(), want)
fmt.Printf("matrix:\s%d\ncolumns,\s%d\nrows\n", m.ncol, len(m.rows))
fmt.Printf("Algorithm\sX:\s%d\nodes,\s%d\solutions\n", tr.nodes, tr.solutions)
xc := cells.NewXCC()
xc.Debug = true
res := xc.Dance(strings.NewReader(dlxText(m)))
got := 0
for range res.Solutions {
    got++
}
fmt.Printf("ssxcc:\s%d\solutions,\s%d\nodes\s\s(agrees:\s%v)\n",
    got, xc.Nodes(), got == want)
```

This code is used in section 4.

33. The exhaustive sweep is the verdict. Every ordered tree up to a given size, every way of designating its leaves, both readings of answer 30. Ten nodes gives 258,564 pairs; the printed answer fails on one of them and the erratum on none.

$\langle$ Try every small tree 33 $\rangle \equiv$

```
total, bad := 0, 0
for n := 1; n ≤ *upto; n++ {
    for _, t := range allTrees(n) {
         $\langle$  Try every way of designating leaves as solutions 34  $\rangle$ 
    }
}
fmt.Printf("errata=%v:\s%d\s(tree,\s\marking)\s\pairs,\s%d\sfailures\n",
    errata, total, bad)
```

This code is used in section 4.

34. There are 2^L markings of a tree with L leaves, and the bits of a counter name them.

⟨ Try every way of designating leaves as solutions 34 ⟩ $\equiv$

```

ls := leaves(t)
for mask := 0; mask < 1 << len(ls); mask ++ {
  c := clone(t)
  cls := leaves(c)
  for i := range cls {
    cls[i].sol = mask & (1 << i) ≠ 0
  }
  total ++
  ⟨ Judge this tree and marking 35 ⟩
}
```

This code is used in section 33.

35. Four things must hold. The backtrack tree must have the shape we asked for; it must have as many nodes as the tree has; step X3 must never have found a tie; and the solutions must be exactly the leaves we starred. On top of that, the duplicate rows in the matrix must appear precisely when two designated leaves are siblings, which is the remark at the end of answer 30 read as a claim about this construction.

⟨ Judge this tree and marking 35 ⟩ $\equiv$

```

tr := algX(makeMatrix(c))
want := 0
for _, l := range leaves(c) {
  if l.sol {
    want ++
  }
}
ok := sameShape(c, tr.shape) ∧ tr.nodes ≡ c.size() ∧
tr.ties ≡ 0 ∧ tr.solutions ≡ want ∧
(tr.dups > 0) ≡ hasSiblingSolutions(c)
if ¬ok {
  bad ++
  if bad ≤ 12 {
    fmt.Printf("□□FAIL□%s\n", c)
  }
}
```

This code is used in section 34.

36. Before the sweep was written I ran the same judgment on random trees, which is how the construction was first believed. It is kept because it reaches sizes the sweep never will—a few thousand trees of twenty-odd nodes cost less than all the trees of eleven.

⟨ Try random trees 36 ⟩ $\equiv$

```

rnd := rand.New(rand.NewSource(*seed))
bad, most := 0, 0
for i := 0; i < *trials; i++ {
    t := randTree(rnd, *budget)
    if *sol {
        markSolutions(t, rnd, 0.4, *sibs)
    }
    ⟨ Judge one random tree 37 ⟩
}
fmt.Printf("%d trials, %d mismatches, %d most duplicate rows in one matrix: %d\n",
    *trials, bad, most)

```

This code is used in section 4.

37. ⟨ Judge one random tree 37 ⟩ $\equiv$

```
| := algX(makeMatrix(t))
want := 0
for _, l := range leaves(t) {
    if l.sol {
        want++
    }
}
if tr.dups > most {
    most = tr.dups
}
if ¬(sameShape(t, tr.shape) ∧ tr.nodes ≡ t.size() ∧
    tr.ties ≡ 0 ∧ tr.solutions ≡ want) {
    bad++
    if bad ≤ 5 {
        fmt.Printf("MISMATCH tree=%s nodes=%d/%d ties=%d sols=%d/%d shape=%s\n",
            t, tr.nodes, t.size(), tr.ties, tr.solutions, want, tr.shape)
    }
}

|  |

```

This code is used in section 36.

38. Index.

a: [25](#).
algX: [20](#), [28](#), [32](#), [35](#), [37](#).
allTrees: [27](#), [33](#).
append: [8](#), [9](#), [13](#), [14](#), [15](#), [16](#), [23](#), [26](#), [27](#).
b: [7](#), [21](#), [25](#), [30](#).
bad: [33](#), [35](#), [36](#), [37](#).
best: [22](#), [23](#).
Bool: [5](#).
bool: [6](#), [10](#), [12](#), [17](#), [20](#), [23](#), [24](#), [25](#), [26](#).
budget: [5](#), [9](#), [36](#).
build: [13](#), [14](#), [18](#).
Builder: [7](#), [30](#).
byte: [29](#).
C: [13](#), [14](#), [15](#), [16](#).
c: [8](#), [15](#), [22](#), [26](#), [29](#), [30](#), [34](#), [35](#).
cells: [4](#), [32](#).
cj: [14](#).
clash: [23](#).
clone: [26](#), [34](#).
cls: [34](#).
cols: [13](#), [20](#), [21](#), [22](#), [23](#).
contains: [15](#), [17](#), [22](#), [23](#).
copy: [23](#).
d: [9](#).
Dance: [32](#).
Debug: [32](#).
dlxText: [30](#), [31](#), [32](#).
dups: [19](#), [24](#), [35](#), [37](#).
empty: [21](#).
errata: [5](#), [12](#), [13](#), [15](#), [33](#).
f: [27](#).
Fatalf: [4](#), [8](#).
first: [27](#).
fix: [5](#).
flag: [5](#).
Float64: [10](#).
float64: [10](#).
fmt: [24](#), [28](#), [29](#), [30](#), [32](#), [33](#), [35](#), [36](#), [37](#).
forests: [27](#).
Fprintf: [30](#).
got: [32](#).
hasSiblingSolutions: [26](#), [35](#).
hide: [19](#).
hit: [23](#).
i: [20](#), [25](#), [29](#), [34](#), [36](#).
Int: [5](#).
int: [7](#), [9](#), [11](#), [12](#), [13](#), [14](#), [15](#), [16](#), [17](#), [19](#), [20](#), [23](#), [27](#).
Int64: [5](#).
Intn: [9](#).
Ints: [13](#), [15](#), [16](#).
j: [9](#), [15](#).
k: [7](#), [10](#), [14](#), [24](#), [26](#), [27](#).
kids: [6](#), [7](#), [8](#), [9](#), [10](#), [13](#), [14](#), [15](#), [23](#), [25](#), [26](#), [27](#).
 Knuth, Donald Ervin: [1](#).
l: [32](#), [35](#), [37](#).
least: [22](#).
leaves: [26](#), [32](#), [34](#), [35](#), [37](#).
len: [7](#), [9](#), [10](#), [13](#), [20](#), [23](#), [25](#), [26](#), [28](#), [32](#), [34](#).
line: [29](#).
log: [4](#), [8](#).
ls: [34](#).
m: [20](#), [22](#), [23](#), [24](#), [28](#), [29](#), [30](#), [32](#).
main: [4](#).
make: [20](#), [23](#), [29](#).
makeMatrix: [18](#), [28](#), [31](#), [32](#), [35](#), [37](#).
markSolutions: [10](#), [36](#).
mask: [34](#).
matrix: [11](#), [18](#), [20](#), [30](#).
mode: [4](#), [5](#).
most: [36](#), [37](#).
n: [7](#), [22](#), [26](#), [27](#), [33](#).
nc: [23](#).
ncol: [11](#), [18](#), [20](#), [28](#), [29](#), [30](#), [32](#).
New: [36](#).
NewReader: [32](#).
NewSource: [36](#).
NewXCC: [32](#).
nextCol: [12](#), [13](#), [14](#), [18](#).
node: [6](#), [7](#), [8](#), [9](#), [10](#), [13](#), [18](#), [19](#), [20](#), [25](#), [26](#), [27](#).
Nodes: [32](#).
nodes: [19](#), [20](#), [28](#), [32](#), [35](#), [37](#).
nr: [23](#).
ok: [35](#).
os: [31](#).
out: [26](#), [27](#).
p: [10](#), [16](#).
p0: [13](#), [14](#), [15](#), [16](#).
p1: [13](#), [14](#), [15](#), [16](#).
p2: [13](#), [14](#), [15](#).
Parse: [5](#).
parse: [8](#).
pos: [8](#).
Printf: [28](#), [29](#), [32](#), [33](#), [35](#), [36](#), [37](#).
r: [15](#), [16](#), [22](#), [23](#), [24](#), [29](#), [30](#).

r2: [23](#).
rand: [9](#), [10](#), [36](#).
Rand: [9](#), [10](#).
randTree: [9](#), [36](#).
rec: [20](#), [23](#).
res: [27](#), [32](#).
rest: [9](#), [27](#).
rj: [14](#).
rnd: [9](#), [10](#), [36](#).
rows: [11](#), [13](#), [14](#), [15](#), [16](#), [18](#), [20](#), [22](#), [23](#), [24](#), [28](#), [29](#),
[30](#), [32](#).
sameShape: [25](#), [28](#), [35](#), [37](#).
seed: [5](#), [36](#).
seen: [24](#).
self: [20](#), [21](#), [23](#).
shape: [19](#), [20](#), [28](#), [35](#), [37](#).
sibOK: [10](#).
sibs: [5](#), [36](#).
size: [7](#), [9](#), [28](#), [32](#), [35](#), [37](#).
sol: [5](#), [6](#), [7](#), [8](#), [10](#), [13](#), [15](#), [21](#), [25](#), [26](#), [32](#), [34](#), [35](#), [36](#),
[37](#).
Solutions: [32](#).
solutions: [19](#), [21](#), [28](#), [32](#), [35](#), [37](#).
sort: [13](#), [15](#), [16](#).
spec: [5](#), [8](#), [28](#), [31](#), [32](#).
Sprint: [24](#).
Stdout: [31](#).
String: [5](#), [7](#), [30](#).
string: [7](#), [8](#), [24](#), [30](#).
strings: [7](#), [30](#), [32](#).
sub: [14](#), [15](#).
t: [7](#), [8](#), [9](#), [10](#), [13](#), [14](#), [15](#), [18](#), [26](#), [28](#), [32](#), [33](#), [34](#), [36](#),
[37](#).
tail: [27](#).
ties: [19](#), [22](#), [28](#), [35](#), [37](#).
total: [33](#), [34](#).
tr: [20](#), [21](#), [22](#), [24](#), [28](#), [32](#), [35](#), [37](#).
trace: [19](#), [20](#).
tree: [8](#), [28](#), [31](#), [32](#).
trials: [5](#), [36](#).
upto: [5](#), [33](#).
used: [10](#).
want: [32](#), [35](#), [37](#).
WriteByte: [7](#), [30](#).
WriteString: [7](#), [31](#).
x: [17](#), [23](#).
xc: [32](#).
xs: [17](#).

- ⟨ Append the rows of type (i) 15 ⟩ Used in section 13
- ⟨ Append the rows of type (ii) 16 ⟩ Used in section 13
- ⟨ Ask `ssxcc` for a second opinion 32 ⟩ Used in section 4
- ⟨ Branch on every option covering it 23 ⟩ Used in section 20
- ⟨ Choose the item of minimum `LEN` 22 ⟩ Used in section 20
- ⟨ Count duplicate options 24 ⟩ Used in section 20
- ⟨ Declarations 6 ⟩ Used in section 4
- ⟨ Functions 7 ⟩ Used in section 4
- ⟨ Judge one random tree 37 ⟩ Used in section 36
- ⟨ Judge this tree and marking 35 ⟩ Used in section 34
- ⟨ Print the matrix as an array of bits 29 ⟩ Used in section 28
- ⟨ Read the command line 5 ⟩ Used in section 4
- ⟨ Recurse into the subtrees 14 ⟩ Used in section 13
- ⟨ Reproduce the printed matrix 28 ⟩ Used in section 4
- ⟨ Try every small tree 33 ⟩ Used in section 4
- ⟨ Try every way of designating leaves as solutions 34 ⟩ Used in section 33
- ⟨ Try random trees 36 ⟩ Used in section 4
- ⟨ Visit a solution if nothing is left 21 ⟩ Used in section 20
- ⟨ Write the problem as `DLX` text 31 ⟩ Used in section 4

Backtrack Trees to Order

	Section	Page
Introduction	1	1
Trees	6	3
The construction	11	6
Algorithm X, taken literally	19	9
Judging the result	25	12
Running it	28	15
Index	38	19